

1. Дисциплина: Языки программирования Тема: Многопоточность в C++ ФИО: Самошкин Егор Вадимович Группа: СКБ-251
2. Постановка задачи

Необходимо реализовать код на языке **C++ (C++11 и выше)**, который обеспечивает обработку задач несколькими потоками. Для этого необходимо:

1. Реализовать потокобезопасную очередь задач TaskQueue .
2. Создать несколько рабочих потоков worker.
3. Потоки получают задания из очереди и обрабатывают из 1 секунду.
4. Использовать механизмы синхронизации стандартной библиотеки C++.
5. Корректно завершить работу потоков.
6. Реализация программы:

3.1 Заголовочные файлы:

#include - ввод-вывод

#include - Создание потоков

#include - работа с дин. массивом потоков

#include - для создания очереди

#include - для работы с мьютексом

#include <condition_variable> - для работы с condition_variable

#include - для таймера обработки

3.2 Класс TaskQueue:

Хранит очередь, мьютекс для очереди и condition_variable для контроля доступа потоков к данным и предотвращения гонок данных.

Метод push - принимает целое число - номер задачи, блокирует мьютекс, добавляет номер задачи и пробуждает один из ждущих потоков. Как только произойдет выход из этого куска кода, удалится старый Lock и мьютекс освободится (ввиду использования `std::unique_lock` `std::mutex`)

Метод pop - принимает целое число-номер задачи, блокирует мьютекс, если очередь пуста, мьютекс освобождается и поток засыпает. Иначе удаляет номер процесса и аналогично Push освобождает мьютекс. `std::mutex coutMutex`; - мьютекс для корректного вывода.

3.3 Функция worker:

Ждет задачу с помощью цикла и pop (Это НЕ busy-wait, т.к в случае отсутствия задач поток засыпает, а не постоянно крутит условие, строчка `cv.wait...` в `pop()` это обеспечивает) если на вход

получает -1 - поток завершается (задач нет), иначе ждет 1 секунду и выводит сообщение об обработанном процессе.

3.4 Основная функция main:

Создает очередь queue типа TaskQueue, создает динамический массив работников (потоков), их число $N = 4$ (например) по следующей схеме:

В очередь потоков добавляется объект с находящейся в нем функцией, ссылкой на очередь (для этого используется ref), и номером. затем очередь наполняется 20-ю процессами и 4-мя "-1" для каждого потока, чтобы в конце завершиться.

В конце ждем завершения всех потоков с помощью join для каждого потока

7. Ссылка: <https://github.com/yegor1202/hse>

The screenshot shows a VS Code editor with a C++ file named `HW_251_1.cpp` open. The code defines a `TaskQueue` class with a `pop()` method that uses a mutex and a condition variable to manage a queue of tasks. The `main()` function creates a `TaskQueue` object, a vector of 4 worker threads, and a vector of 20 tasks. Each worker thread takes tasks from the queue and processes them. The terminal output shows the execution of the program, with each worker thread processing a task and printing a message like `[Worker-3] обработал задачу 5`. The program ends with a `std::cout` statement.

```

class TaskQueue {
    int pop() {
        cv.wait(lock, [this]() {
            return !tasks.empty();
        });
        int task = tasks.front();
        tasks.pop();
        return task;
    }
};

std::mutex coutMutex;

```

```

yaegor@yegorSamoshkin MINGW64 ~/Desktop/VSCode/HWModule (master)
$ /usr/bin/env c:\Users\yaegor\vscode\extensions\ms-vscode.cpptools-1.30.5-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe --stdin=Microsoft-MIEngine-In-hfqzimb.stk --stdout=Microsoft-MIEngine-Out-1em8ivei.lct --stderr=Microsoft-MIEngine-Error-s543btjl.slw --pid=Microsoft-MIEngine-Pid-5yflxahk.qup --dbgExe=c:\Users\yaegor\gcc\bin\gdb.exe --interpreter=mi

[Worker-3] обработал задачу 5
[Worker-2] обработал задачу 10
[Worker-4] обработал задачу 11
[Worker-1] обработал задачу 9
[Worker-3] обработал задачу 12
[Worker-2] обработал задачу 13
[Worker-1] обработал задачу 15
[Worker-4] обработал задачу 14
[Worker-3] обработал задачу 16
[Worker-2] обработал задачу 17
[Worker-1] обработал задачу 18
[Worker-3] обработал задачу 20
[Worker-4] обработал задачу 19

yaegor@yegorSamoshkin MINGW64 ~/Desktop/VSCode/HWModule (master)
$

```

8. Выводы

Я изучил основы работы с многопоточностью в C++, понял как работает мьютекс, `condition_variable` и что это такое, реализовал в программе модель потокобезопасной очереди.